Templates, primitives, and the planner

Ariadne — knowledge-graph question answering over ATT&CK and CAPEC · results brief, 14 September 2026 · rev. 2
Every figure below is recomputed from judgment files, not quoted from prose. Sources named in the footer.

> **Two questions were on trial.**
>
> **1. Can the template engine be retired?** It answers only questions someone wrote a template for — it is not an open-ended process, so an arbitrary question cannot be fed to it. The replacement composes answers from five universal primitives instead.
>
> **2. Can a small (2B) model make answers readable for humans?** Its job is presentation, not retrieval — turn a correct structured answer into clear prose without disturbing a single fact.

Headline

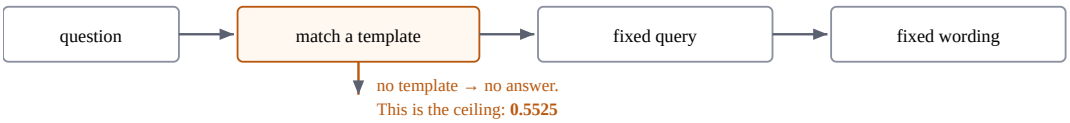
Both answers are yes, with one caveat that decides how the result may be used. The open-ended engine beats the template engine by **+0.348** on questions nothing had ever seen, with zero fabricated mappings, and it is not meaningfully slower. The 2B model rewrites prose without touching facts — provably, across 284 answers. The caveat is that both headline arms replay a **human-authored retrieval plan**. Letting the system plan for itself costs **−0.381** — and at that plan source, which is what the deployed workbench runs, **the engine advantage shrinks to +0.026 and is no longer distinguishable from zero.** The primitive engine is what makes the system open-ended; it is not yet a drop-in replacement.

Arm	What it is	Correctness	Fabricated mappings	s / answer
`deterministic`	hand-written templates	0.5525 ±0.0197	0	0.04
`generic_composed`	**5 primitives, no model anywhere**	**0.9005 ±0.0102**	**0**	**0.05**
`generic`	5 primitives + LLM narrator	0.8991 ±0.0103	0	66.82

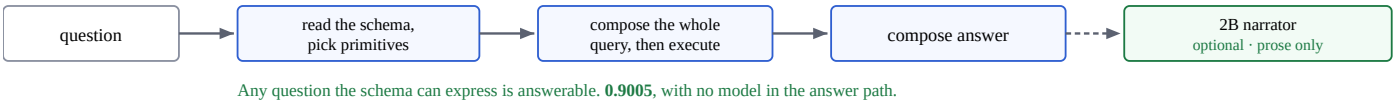
282 questions, seed 10140325, authored blind and sealed before use — never asked, never graded. Both grading positions, counterbalanced. Latency at one worker on a matched 20-question slice.

How the two engines differ

Template engine — closed set



Open-ended engine — five primitives, composed per question



Any question the schema can express is answerable. **0.9005**, with no model in the answer path.

The five primitives — `describe`, `generic_neighbors`, `generic_walk`, `generic_chain`, `generic_bridge` — are the whole vocabulary. The engine reads a precomputed schema registry (which relation types connect which node labels, and in which direction), composes the entire query before touching the database, and executes once. Nothing about the graph's contents is needed to plan the shape of the query, which is why it works on questions no one anticipated.

Result 1 — the template engine is dominated

The decisive test was a **sealed set**: 282 questions generated at a fixed seed, authored blind against briefs, and cryptographically sealed before any arm saw them. A self-test confirmed the sealing tool would refuse a previously used bank — it did refuse the old holdout, on 3,456 prior judgments.

Dimension	deterministic	generic_composed	Verdict
Correctness, unseen questions	0.5525	**0.9005**	−0.348 against templates
Fabricated mappings (sealed set)	0	0	tie
Fabricated mappings (regression)	3	**0**	templates still fabricate
Wrong-entity rate	0.0950 shipped — lowest of 7 reference captures		no regression
Latency, single user	0.04 s	0.05 s	0.01 s — not a fast path
Questions it can accept	template must exist	**any**	the original motivation

> **There is no measured dimension on which the template engine wins.** The one argument that could have survived a coverage loss — that templates are the fast path — is worth a hundredth of a second per answer. Retiring it costs nothing and removes a maintenance surface that grows with every new question type.

The result holds on a second, independent question bank

The same three arms were re-run over a separate 288-question bank on a different night. Both engines land within **0.003** of their sealed-set scores.

Arm	Sealed set (282)	Second bank (288)	Difference
`deterministic`	0.5525	0.5443	0.008
`generic_composed`	0.9005	0.8987	0.002
`generic`	0.8991	0.9014	0.002

The second bank had been used before, so it is reported as a regression check and never as a capability figure. Its value is the agreement, not the level: the result generalises rather than fitting the set it was tuned on.

Result 2 — the 2B model does its job, and only its job

The small model was never asked to improve correctness. It was asked to make a correct structured answer read well for a human. The test is therefore **fidelity**, not accuracy: did the prose change any fact?

Check, across 284 answers	Result	Reading
Claims byte-identical to the model-free control	**284 / 284**	**no fact was altered**
Changes to entity identifiers	0	no entity substitution
Changes to citations	0	evidence links intact
Changes to verdicts	0	no abstention overturned
New unsupported statements	0	nothing invented
Effect on retrieval coverage	+0.0000	exactly zero, zero spread
Fallback rate (transport failures)	0.0282	within the 0.14 bar

> **This is the result the presentation goal needed.** A narrator that cannot move a correctness metric is a narrator that cannot corrupt an answer. The +0.0000 is not a disappointing null — it is the safety property, measured from two directions independently.

What it costs, and the one thing it cannot do

The seam costs time, not accuracy: **66.8 s per answer against 0.05 s** — roughly 1,340×, for a correctness difference of −0.0014 that sits inside the grader's noise. So prose generation belongs **on demand, per answer**, not in the default path.

The same 2B model was also tried at the harder seam — **drafting database queries**. It is not usable there: **0 working drafts out of 17**, and quadrupling its token budget changed the failure mode without changing the outcome. Query construction stays deterministic.

Result 3 — the caveat that reframes every figure above

Both 0.90 arms answer using a **human-authored retrieval plan**, replayed. The planner does not run. When the system is made to produce its own plan, retrieval coverage falls by **−0.381** (paired, n = 284, SE 0.0262 — fourteen standard errors, not noise).

Gold coverage, 284 questions	no model	2B narrator	model effect
authored plan	**0.8611**	**0.8611**	**+0.0000**
system plans for itself	0.4797	0.4824	+0.0026
plan effect	−0.3814	−0.3787	interaction +0.0027

All four cells run and paired per question — the fourth had never been measured. The two variables do not interact, so the corners are not misleading each other.

On **77 of 284 questions the planner retrieved nothing where the authored plan retrieved records. The reverse never happened once.** The loss is worst on chained and cross-catalogue questions (−0.50 and −0.55), which are exactly the questions with the most plan structure to get wrong.

And the engine advantage does not survive that planner

Crossing both factors — which engine, which plan source — on the same 284 questions answers the question the deployment decision actually turns on. The workbench runs the real planner, so the right-hand column is where it stands.

Gold coverage	authored plan	real planner *(the workbench)*
hand-written templates	0.6428	**0.4536**
schema-derived primitives	**0.8611**	0.4797
primitives − templates	**+0.2184** ***separable***	**+0.0261** ***not separable***

Interaction −0.1923. Unlike the model seam, these two factors interact strongly — so the corner cells mislead on their own.

A bad plan costs the primitive engine roughly twice what it costs the templates (−0.3814 against −0.1892), and under the real planner it returns nothing on 85 of 284 questions against the templates' 55. A template constrains the shape of the query, so a poor decomposition still lands somewhere defensible; a primitive follows the plan literally, so a poor decomposition becomes a poor traversal. **The generality that lets primitives answer anything is the same property that makes them faithful to a bad instruction.**

The effect also **changes sign by question type**: templates win easy (−0.1586) and adversarial (−0.1389), primitives win hard (+0.0659) and medium (+0.1127). The +0.0261 aggregate is a cancellation, not an agreement.

> **What this means, stated precisely.** 0.90 is a measured ceiling for **retrieval and composition given a correct plan**. It is a real ceiling — it is what the engine achieves on questions nobody wrote a template for — but it is not yet an end-to-end figure for a system answering arbitrary questions unaided. The open-endedness claim is proven for the *engine* and not yet for the *planner*.
>
> One limit on the −0.381 itself: the self-planning arm also inherits tighter per-plan bounds, so "chose a worse decomposition" is not yet separated from "chose tighter limits". That separation needs a third arm and is recorded as an open issue rather than assumed away.

What follows concretely

Decision	Supported?	Evidence
Retire the template engine *outright*	**No**	at the workbench's own plan source the gap is +0.0261 and not separable, with 85 empty answers against 55
Keep the primitive engine as the open-ended path	**Yes**	+0.2184 given a good plan; and templates cannot accept an unanticipated question at all
Route between them rather than replace	**Yes**	the effect changes sign by question type — templates win easy and adversarial, primitives win hard and medium
Ship the answer path with no model in it	**Yes**	`generic_composed` 0.9005 with every LLM seam off
Use the 2B model for human-facing prose	**Yes, on demand**	284/284 facts preserved; 1,340× latency forbids it as default
Use the 2B model to draft queries	**No**	0 working drafts of 17; budget increase does not help
Claim end-to-end open-ended QA at 0.90	**Not yet**	planner costs −0.381; that gap is the next experiment

Recommended shape. Two engines with a router, not one engine — and no model in the fact path either way. The primitive path is what makes the system open-ended and is the only path that can take an unanticipated question at all; the template path stays because at the planner the workbench actually runs it is measurably better on routine and adversarial questions. The 2B narrator becomes a presentation layer the reader opts into per answer.

The real target is the planner. It costs −0.1892 to templates and −0.3814 to primitives, and it is the largest term in every cell of that table. Fixing it lifts both engines and is what would make a straight swap worth making: the +0.2184 is what becomes available if the planner reaches the quality an authored plan has. The engineering question is no longer "can this generalise" — it is "can the planner reach the ceiling the engine already demonstrates".

How much to trust these numbers

- **A pre-registered kill gate that fired.** The first judged pass produced this project's first non-zero fabrication count. One defect explained it — a scope line asserted from the intended filter rather than from what the query actually did — and the fix made the claim one-way: it can withhold a true statement, never assert a false one.
- **Five composer defects found and repaired** during the run; four were caught before any result was read.
- **A size-correlated grading bias, found and closed three times.** Answers that failed to grade were always the largest, never a random sample. Recovering eight moved the LLM arm *down*, 0.9043 → 0.8991. Every figure here is over the complete set.
- **One figure withdrawn and corrected here.** The narrator's latency cost was recorded as "90×"; that divided a per-answer rate by a 20-question total. Re-measured: ~1,340×. The erroneous value is retained as withdrawn, not deleted.
- **Grading:** capability figures average two counterbalanced positions. Single-tier differences below ~0.06 are indistinguishable from grader noise and are not claimed.

Sources — `EXP-2026-09-018` (repair, paired control, sealed set, regression, latency, T6) and `EXP-2026-09-019` (endpoint, planner cost, 2B narrator and specialist seams, model×plan 2×2) and `EXP-2026-09-020` (engine×plan-source 2×2, the deployment question). All values recomputed from judgment sidecars via each experiment's `data/*.json`; none transcribed from prose. Grading by a single model family on one endpoint — a threat to validity recorded in both protocols.